EXPERIMENT 16:TO IMPLEMENT FEED FORWARD NEURAL NETWORK

Aim

To implement a Feed Forward Neural Network (FFNN) for classification and understand how input data is passed through the input, hidden, and output layers to produce the final result.

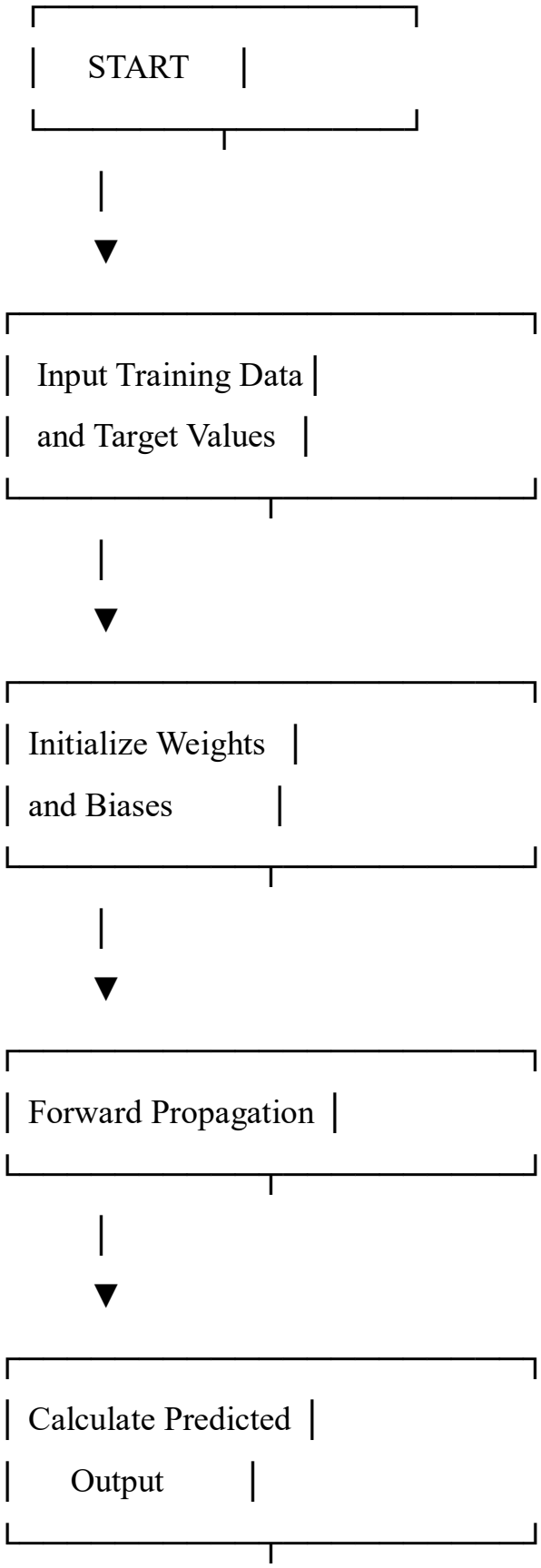
Objective

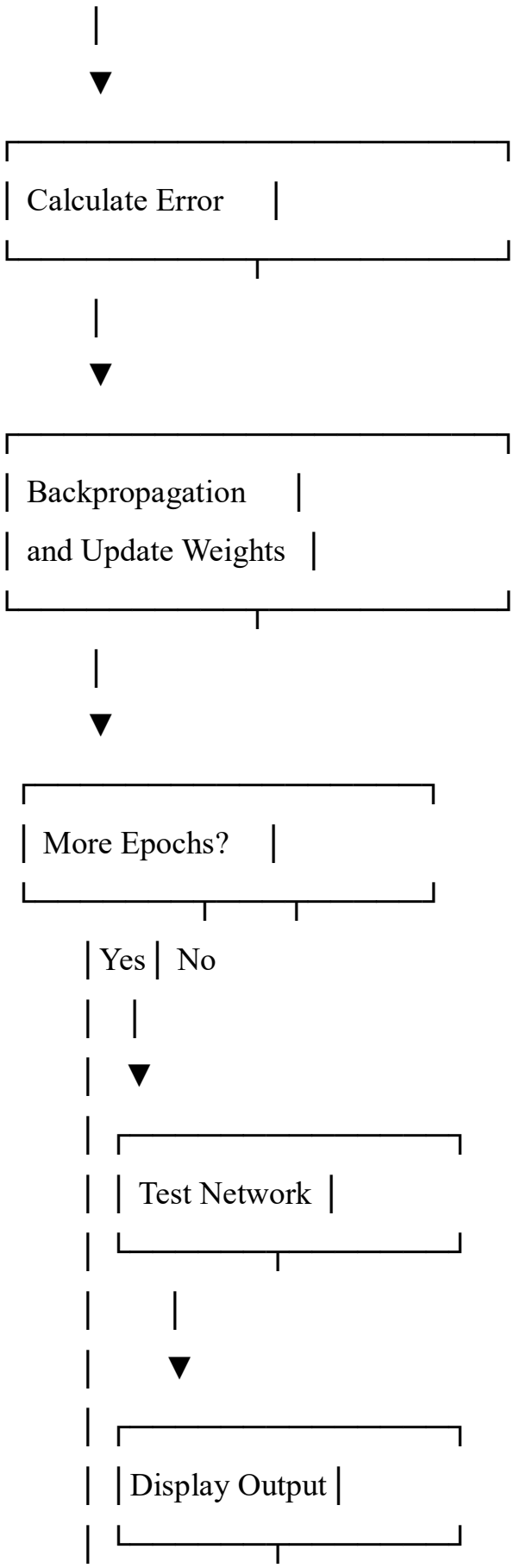
- To understand the structure of a Feed Forward Neural Network.
- To implement a neural network using Python.
- To train the network using the backpropagation algorithm.
- To classify input data based on the trained network.
- To understand how weights and biases are updated during training.

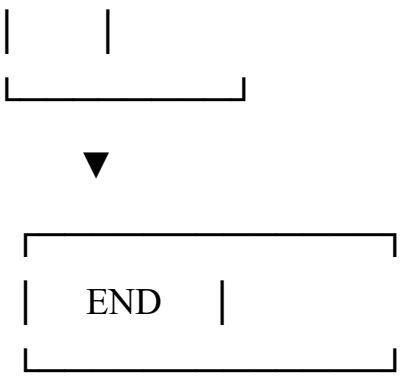
Algorithm

1. Start the program.
2. Initialize the input data and target output.
3. Initialize the weights and biases randomly.
4. Pass the input values through the input layer.
5. Calculate the weighted sum at the hidden layer.
6. Apply an activation function to the hidden-layer output.
7. Pass the hidden-layer output to the output layer.
8. Calculate the predicted output.
9. Calculate the error between the predicted output and target output.
10. Apply backpropagation to calculate the required weight and bias updates.
11. Update the weights and biases.
12. Repeat steps 5–11 for the specified number of epochs.
13. Test the trained network with input data.
14. Display the predicted output.
15. Stop the program.

Flowchart


START
Input Training Data
and Target Values
Initialize Weights
and Biases
Forward Propagation
Calculate Predicted
Output


Calculate Error
Backpropagation
and Update Weights
More Epochs?
Yes
No
Test Network
Display Output



Code

```
# Feed Forward Neural Network
# Implemented using Python and NumPy

import numpy as np

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))



# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)



# Input data
# XOR problem
X = np.array([
    [0, 0],
    [0, 1],
```

```
    [1, 0],
    [1, 1]
])

# Target output
y = np.array([
    [0],
    [1],
    [1],
    [0]
])


# Set random seed
np.random.seed(1)

# Initialize weights
input_hidden_weights = np.random.uniform(
    size=(2, 2)
)

hidden_output_weights = np.random.uniform(
    size=(2, 1)
)

# Initialize biases
hidden_bias = np.random.uniform(
```

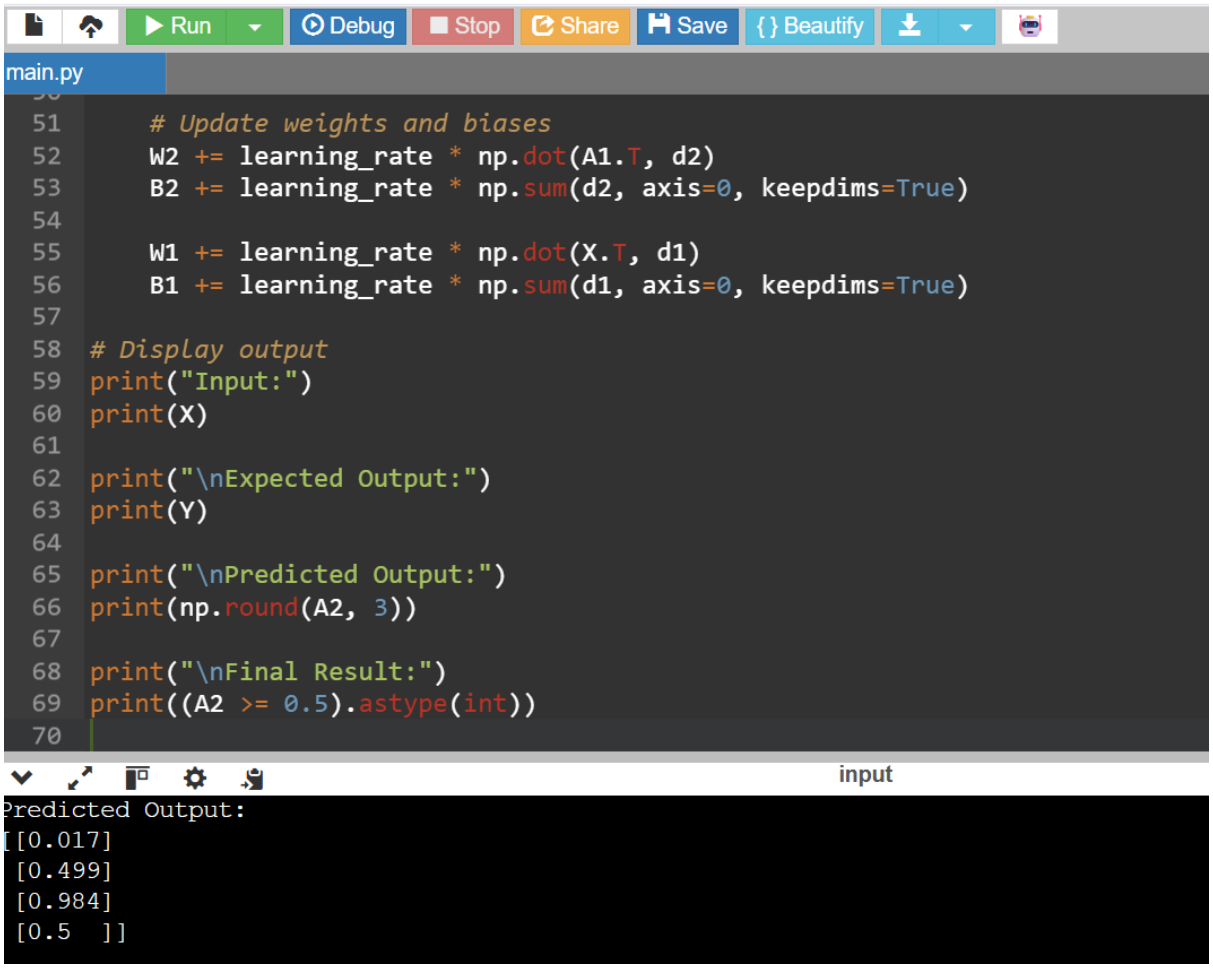
```
    size=(1, 2)
)

output_bias = np.random.uniform(
    size=(1, 1)
)


# Learning rate
learning_rate = 0.5

# Number of training iterations
epochs = 10000


# Training
for epoch in range(epochs):

    # ------------------------
    # Forward Propagation
    # ------------------------

    hidden_input = np.dot(
        X, input_hidden_weights
    ) + hidden_bias

    hidden_output = sigmoid(hidden_input)
```

```
output_input = np.dot(
    hidden_output,
    hidden_output_weights
) + output_bias

predicted_output = sigmoid(output_input)



# -------------------------
# Calculate Error
# -------------------------

error = y - predicted_output



# -------------------------
# Backpropagation
# -------------------------

output_delta = (
    error *
    sigmoid_derivative(predicted_output)
)

hidden_error = np.dot(
    output_delta,
```

```
    hidden_output_weights.T
)

hidden_delta = (
    hidden_error *
    sigmoid_derivative(hidden_output)
)



# ------------------------
# Update Weights and Biases
# ------------------------

hidden_output_weights += (
    np.dot(hidden_output.T, output_delta)
    * learning_rate
)

output_bias += (
    np.sum(output_delta, axis=0, keepdims=True)
    * learning_rate
)

input_hidden_weights += (
    np.dot(X.T, hidden_delta)
    * learning_rate
)
```

```
    hidden_bias += (
        np.sum(hidden_delta, axis=0, keepdims=True)
        * learning_rate
    )


# ------------------------
# Testing the Neural Network
# ------------------------

print("Input:")
print(X)

print("\nTarget Output:")
print(y)

print("\nPredicted Output:")
print(np.round(predicted_output, 3))

print("\nFinal Classification:")

for value in predicted_output:
    if value[0] >= 0.5:
        print(1)
    else:
        print(0)
```

Output



Result

The Feed Forward Neural Network was successfully implemented using Python. The network learned the XOR pattern and produced outputs close to the expected target values.

Conclusion

The Feed Forward Neural Network successfully learned the relationship between the input and output data through forward propagation and backpropagation. The trained network was able to correctly classify the XOR inputs, demonstrating the basic working principle of a neural network.